Global Illumination for Fun and Profit

Josiah S. Carberry, Ed Grimley, and Martha Stewart

Fig. 1: In the Clouds: Vancouver from Cypress Mountain. Note that the teaser may not be wider than the abstract block.

Abstract—Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum. A free copy of this paper and all supplemental materials are available at https://OSF.IO/2NBSG.

Index Terms—Radiosity, global illumination, constant time

1 Introduction

Tree structures are ubiquitous across various domains, including organizational hierarchies, computational execution plans, program architectures, and file systems. In real-world scenarios, these tree structures often evolve over time. For instance, organizations restructure to align with business needs, execution plans are iteratively refined from initial drafts, and program structures evolve through maintenance and refactoring. Consequently, understanding and visualizing the evolution of tree structures over time holds significant analytical and decision-making value.

This study focuses on evolving trees, where both data and topology co-evolve over time. We define the changes between consecutive time points as the tree's evolution process. Tree evolution exhibits three prominent characteristics: **Locality**: Within short time intervals, changes are typically localized, while major transformations often align with pivotal events, such as organizational overhauls or plan resets. **Wide variability**: The number of nodes and hierarchical depth can vary dramatically across time, with scales ranging from tens to tens of thousands of nodes. **Long sequences**: The timeline may span hundreds or thousands of snapshots, necessitating the ability to capture both long-term global trends and fine-grained local details. To address core questions of how and why a tree evolves, analysts must fluidly navigate between global patterns (e.g., periodic structural reorganizations, evolutionary rhythms, or load shifts) and local trajectories (e.g., the creation, deletion, migration, or attribute changes of individual nodes).

Several visualization methods exist for dynamic tree structures: **Stable treemap**: These methods enhance layout and transition strategies to preserve spatial and topological consistency across time, often using animations to depict changes. While visually coherent and engaging, animation-based approaches are less effective for memorizing processes and are unsuitable for complex, long-sequence analyses. **Stream Graph**: The methods encode aggregated metrics (e.g., subtree size or weight) through stream widths, utilizing nested relationships among flows to represent hierarchical structures. This approach visualze the trends of metrics and evolution of structure in a static view ideal for long-term analysis. However, they often obscure topological and hierarchical semantics, making it challenging to identify where structural changes occur or to accommodate all types of tree evolution. **Graph sequence**. These combine graph and sequence visualization by discretizing time series into snapshots, employing static tree representations (e.g., treemaps, icicle plots, or node-link diagrams) organized sequentially for cross-time comparison and tracking. These methods can prioritize either topological or hierarchical structures based on requirements. However, existing methods designed for general graph data are not optimized for effectively visualizing the evolution process of tree structures or tracking the trajectories of significant nodes.

- *Josiah Carberry is with Brown University. E-mail: jcarberry@example.com*
- *Ed Grimley is with Grimley Widgets, Inc. E-mail: ed.grimley@example.com.*
- *Martha Stewart is with Martha Stewart Enterprises at Microsoft Research. E-mail: martha.stewart@example.com.*

Manuscript received xx xxx. 201x; accepted xx xxx. 201x. Date of Publication xx xxx. 201x; date of current version xx xxx. 201x. For information on obtaining reprints of this article, please send e-mail to: reprints@ieee.org. Digital Object Identifier: xx.xxxx/TVCG.201x.xxxxxxx

To address these challenges, we propose NAME, an interactive visual analytics system for tree evolution. This system enables users to systematically explore why a tree evolves, when and where changes occur, and who is affected by these changes. The approach is built upon a graph sequence visualization framework, with designs tailored to the key characteristics of tree evolution. First, we introduce a set of extensible tree evolution event primitives (e.g., node birth/death, attribute mutation, subtree migration/reconnection, subtree merging/splitting, and weight mutation) along with corresponding automated detection and matching strategies. By extracting events, we compress long snapshot sequences into event sequences, significantly reducing the complexity of long-sequence analysis while providing a consistent semantic foundation for subsequent phase segmentation and pattern recognition. Next, we design a structurally stable node-link visual encoding to represent the evolution process over continuous time periods. Combined with graph difference visualizations, we enhance annotations for critical changes, enabling analysts to quickly identify significant structural shifts and anomalies while preserving topological semantics and tracing the context of changes. Finally, we propose an ego-centric node highlight method to intuitively display the detailed evolution processes of individual nodes, supporting multi-level sequence overviews and phase segmentation.

The major contributions of this work can be summarized as follows:

- We conduct an in-depth analysis and survey of the requirements for tree evolution processes, distilling the fundamental evolution patterns.
- We propose a novel tree visualization method capable of supporting continuous graph change processes.
- We develop a suite of analysis tools that address multi-level analytical needs, including sequence analysis and ego-centric node analysis.
- We present case studies drawn from database systems and LLM (Large Language Model) trees to demonstrate the system's efficacy.

The remaining parts of the paper are organized as follows:

2 Related work

2.1 Stable Tree Layouts

A common approach to visualize evolving trees is to use layouts that remain as spatially stable as possible over time. In other words, the visual positions of nodes are preserved or minimally changed between successive time steps, helping users maintain a consistent mental map of the hierarchy as it evolves. Both node-link diagrams and space-filling tree visualizations (such as treemaps) have been adapted with this goal in mind. Treemaps, in particular, are frequently used for dynamic hierarchies because they maximize screen space usage (scaling to thousands of items) compared to node-link layouts [18]. However, traditional treemap algorithms can be unstable under change: even small updates in the data can trigger disproportionately large shifts in the layout. Reducing this instability is a central challenge for temporal treemap design [18]. Early work explored how to visualize hierarchical changes with treemaps [17], and since then numerous techniques have been proposed to improve layout coherence over time. For example, some methods impose a fixed ordering or spatial subdivision that remains constant, such as using Hilbert or Moore space-filling curves to define treemap cell positions [16]. This ensures that each item's rectangle moves little or not at all between time steps, at the cost of some flexibility in optimizing aspect ratios or exact placement. Other algorithms take optimization approaches which incrementally adjust the layout for changes while preserving the relative neighborhood structure of nodes as much as possible [15]. Similarly, in the realm of Voronoi (cell-based) treemaps, researchers have developed stable layout schemes – for instance, Hahn et al. [12] describes a method to maintain a consistent Voronoi treemap configuration for varying hierarchies in software evolution data. These techniques collectively aim to balance two competing objectives: maintaining temporal consistency (so that users can track elements over time) versus achieving spatial clarity and optimal use of space at each time step. In practice, a compromise is often necessary. Stable layout algorithms may intentionally sacrifice some ideal aesthetic (e.g., perfect aspect ratios or minimal blank space) to avoid jarring the viewer with drastic re-layouts. They work best when changes are incremental; if a tree undergoes a major reorganization (for example, a large subtree is added or removed), even the most stable algorithm will still exhibit noticeable changes or must significantly distort the layout to accommodate the update. Nonetheless, when successful, stable tree layouts allow viewers to perform "mental animation" – comparing states over time by seeing that each node remains in a predictably similar screen location from one time step to the next.

Node-link tree diagrams can likewise be made more stable, though this area is somewhat less explored than treemaps in the literature. The straightforward approach for node-link layouts is to use incremental or constrained graph drawing algorithms that update an existing layout slightly rather than recomputing from scratch for each time step. For example, one can keep the parent–child structure drawn in a similar fashion and insert new nodes in a way that only locally perturbs the diagram. This preserves the overall topology and node ordering of previous steps, helping to maintain the viewer's frame of reference. In dynamic graph visualization, such mental map preservation strategies (e.g. anchoring certain nodes or gradually interpolating changes) are known to improve readability over animations [11]. For trees, an added advantage is that the inherent hierarchy provides an ordering that can be held constant (for instance, siblings can remain in the same sorted order across time). Some systems explicitly prioritize this; for instance, if a tree grows by adding nodes, a stable node-link layout might place the new nodes in reserved space or at the end of an ordering to minimize shuffling of existing branches. The limitation, however, is analogous to treemaps: over time, a strictly stable node-link layout can become sub-optimal (with uneven spacing or long edges) as the tree evolves. Thus, designers often incorporate smooth animated transitions along with partial re-layouts to help users track moving nodes [10].

In summary, stable layout approaches (whether space-filling or node-link) are characterized by preserving visual continuity. They enable side-by-side time comparison and sequence analysis by ensuring that each element of the tree can be "tracked" over time in a consistent location, thereby reducing the cognitive effort needed to follow the structure's evolution.

2.2 Stream-Based Visualizations

nspired by streamgraphs and flow diagrams, another approach represents an evolving tree as a set of streams or bands that flow along a time axis. These techniques forego drawing an explicit node-link diagram at each time in favor of an integrated depiction in which time is typically on one axis (e.g. horizontal), and the hierarchical components are depicted as continuous regions whose size or position changes over time. The classic example is ThemeRiver, which pioneered the idea of stacking time-series data as flowing "rivers" to show thematic changes [13]. A stacked graphs based approach further refined the visual aesthetics of these flows, for instance, reducing abrupt wiggles and ensuring smooth transitions, which further improves the readability of trends in the data [3]. When applying such stream-based visualization to hierarchical data, the key challenge is to convey the tree's multi-level structure within the flowing layout. Several approaches have been developed to do this. One straightforward method is to use color coding or layering to differentiate contributions of different subtrees within a single stream: for example, a top-level branch of the tree could be represented as an area whose segments are colored to represent its child nodes [14]. This way, the overall outline of the stream shows the total value for that branch over time, while the colored subdivisions indicate how its sub-components contribute at each moment. Another strategy is to draw a small accompanying tree or icicle plot at discrete intervals along the stream to give a structural context [14]. Moreover, Multi-Stream [5] takes a multi-resolution approach: it visualizes hierarchical time series with a streamgraph that users can interactively "drill down" or aggregate, effectively zooming in on particular branches of the hierarchy in the time dimension. This provides a way to manage complexity by showing a coarse overview stream for the entire tree, while allowing exploration of finer details on demand. Across all these variants, a

unifying principle is that the evolution of the data is encoded in smooth, time-aligned shapes rather than as a series of separate snapshots. A branch growing or shrinking in importance is seen as a stream band thickening or thinning, and temporal patterns like cyclic fluctuations or sudden changes become visually apparent as undulations in the flow. Importantly, because the streams are typically ordered consistently (and often vertically aligned in a meaningful way), the display is inherently temporally stable: one can trace a given category or node's "flow" from left to right without it jumping around to different positions as time advances.

Stream-based visualizations are very effective for conveying quantitative changes and high-level trends in an evolving hierarchy, but they have some trade-offs. One limitation is that most such approaches assume the topology of the tree remains unchanged while the node values vary over time [14]. In other words, they usually handle cases where the set of nodes (and the parent-child relations) are fixed, and only attributes like size or weight change. In these scenarios, a static ordering of streams can be maintained. When the hierarchy itself changes – nodes appearing, disappearing, splitting or merging – it becomes harder to encode that in a streamgraph without introducing discontinuities or confusing the viewer. Some research has begun to address this by incorporating branching and merging flows to indicate structural changes. For example, Cui et al. [6] visualized the evolution of hierarchical topics in a document collection by using a storyline-like flow: when a topic split into subtopics, the stream representing that topic forked into multiple strands, and when topics merged, their streams converged. Their visualization did not attempt to show the entire tree at full detail; instead, it presented a selected "cut" of the hierarchy (for instance, focusing on mid-level topics) at each time step. This selective approach avoided excessive clutter from showing all levels at once, but it means the user sees only part of the hierarchical structure evolving (e.g., major branches but not every leaf node). Generally, stream-based methods prioritize continuity and overview over precise structural depiction. They excel at questions like "How does the total value of this branch change over time?" or "When did this category become dominant relative to others?" – questions that involve comparing magnitudes and trends, for which the contiguous area encoding is very intuitive. On the other hand, if one needs to know the exact tree structure at a given time or track the detailed reconfiguration of subtree relationships, the stream representation can be lacking. The individual nodes are not distinct points but rather part of a blended region, and parent-child relationships are implied by contiguity or color rather than explicitly drawn. Thus, these designs sometimes supplement the visualization with interactive queries (e.g., mouse over a stream to highlight the corresponding subtree in a separate view) or annotations to clarify structure. In summary, flow-based hierarchical visualizations offer temporal stability and an immediate sense of change dynamics, making them powerful for depicting how values in an existing hierarchy evolve. Yet, they handle structural evolution by simplifying or aggregating it to preserve the clean, continuous visual metaphor of a "river" of information over time. They stand in contrast to approaches that show each moment in detail, instead providing an integrated narrative of change that is easy to follow but somewhat abstracted from the exact tree topology.

2.3 Graph Sequence Approaches

The third broad approach to depicting evolving trees treats each time step as a separate graph visualization, combining them in sequence to reveal changes. Unlike streamgraphs, these methods do not merge time into one continuous form; instead, they present a series of discrete snapshots of the tree. This can be done in several ways. One option is through animation, where a single view of the tree smoothly transitions to reflect changes at the next time point. At any given moment, the viewer sees a familiar tree diagram (for example, a node-link tree or a treemap) for that time step; as time progresses (either automatically or via a user-controlled slider), nodes and links appear, disappear, or move to new positions to show the next state. Animation coupled with a stable layout algorithm can effectively highlight changes while preserving context, especially if the transitions are interpolated to avoid abrupt jumps. For instance, a system called TimeTree allowed users to scrub a timeline to examine how an organization chart evolves, using a focus+context tree view that updates as the time slider moves [4]. Such animated or interactive sequences leverage humans' ability to perceive motion: a node that moves slowly from one position to another indicates a structural change, and the user's object permanence helps them understand it is the same node in a new context. The downside, noted in evaluations of animated graph visualization, is that viewers may experience cognitive load in remembering the prior state once it has morphed into the new state. If many changes happen at once, or if the layout is not stable, the viewer can become disoriented – a phenomenon sometimes called the "change blindness" problem in animations. This is why careful animation design (e.g., staggering multiple changes, using ghosting or trails for moved elements) and/or supplemental techniques (such as textual change lists or highlights for new and removed nodes) are often recommended when using animation for tree evolution.

Another way to present graph sequences is through small multiples, i.e. a static array of images where each image is the tree at one point in time. In this approach, time is mapped to a spatial dimension: for example, one might show a grid or row of tree diagrams representing successive years, allowing direct visual comparison between them. Small multiples forego the need for memory – since all states are visible simultaneously – and instead rely on the viewer's eye to scan and compare the different snapshots [1, 9]. They demonstrated that with careful design (like consistent alignment and scaling across views), an observer can trace differences in topology or node values across a handful of trees without animation. This approach can scale reasonably well to a moderate number of time steps; studies have shown it is possible to compare on the order of tens of trees at once by dividing the screen into small subviews. However, it quickly meets practical limits. As more snapshots are included, each must be made smaller (to fit on the screen), which can render detailed labels or small nodes illegible. Moreover, with too many panels, the task of comparison becomes arduous – the viewer may have to jump back and forth across distant panels to track a node's history. For this reason, small-multiple displays of dynamic trees often focus on a limited set of key frames (e.g., yearly snapshots, or "before vs. after" comparisons) or provide interactive means to choose which time slices to view side by side. An interactive variant might let the user select any two or three time points to compare directly [7], or dynamically generate a small multiple for a sliding window of time [8]. In summary, the snapshot sequence approach preserves the full detail of each time step's structure by essentially treating each one as a standalone visualization. This maximizes structural fidelity – each tree is drawn with no compromises for the sake of temporal continuity – but at the cost of requiring the user to integrate the temporal aspect themselves, either by watching an animation or scanning across multiple images.

To better integrate the temporal dimension while still showing each state distinctly, researchers have developed timeline-based layouts that arrange a sequence of tree drawings within a single, combined view. The idea is to lay out the tree multiple times (once per time step) and use the horizontal axis to represent time, effectively creating a 2D tableau where one dimension is the tree structure and the other is time. A representative example is TimeArcTrees [11]. In TimeArcTrees, each time step's tree is drawn as a column of nodes (using a node-link or indented layout vertically), and corresponding nodes across time are aligned on the same row; edges that connect a node's occurrences from one column to the next are drawn as arcs (curved lines) across the intervening space. This produces an informative visual: one can trace a horizontal line (or arc trajectory) to see how a particular node persists over time and where it moves in the hierarchy. For instance, if a node changes parent, the arc connecting its position in one column to its position in the next column will slope upward or downward, revealing the structural move. If it disappears, the arc ends, indicating the node was removed. Burch et al. [2] similarly presented a timeline approach for general dynamic graphs, arranging consecutive graph snapshots side by side and drawing all the changing edges as splines in parallel across them. They referred to a technique called parallel edge splatting, where many edges over time are drawn with translucency so that areas

of frequent change build up a kind of density plot, helping to mitigate over-plotting. The advantage of such timeline-aligned sequences is that they make the temporal correspondences explicit: unlike a pure small multiple where separate images must be mentally correlated, here the visualization itself links the same node's incarnations over time. This can greatly aid in understanding complex evolution—one can literally see the path of a moving sub-tree or the convergence of merging nodes. It effectively combines the strengths of the other approaches: like stable layouts, it keeps a reference for each node across time (through alignment/links), and like small multiples, it shows each time step in full detail.

The timeline approach is powerful but comes with the significant challenge of visual complexity. When a tree has many nodes, drawing each one at every time step, plus connecting lines between them, can clutter the display. If the hierarchy is large and the sequence is long, the result may be an overwhelming hairball of crossing lines. Strategies to cope with this include interactive filtering (e.g., showing connections only for a selected subset of nodes or time range), edge bundling or aggregation techniques to group many parallel evolution paths into a single visual channel, and careful visual encoding (using translucency, thinner lines, or curved routing to reduce overlap). For example, the "edge splatting" technique [2] essentially aggregates multiple similar change paths to highlight general areas of change intensity rather than drawing each change individually. Even with such measures, timeline-based visualizations are usually applied to datasets where the number of changing elements is manageable, or they provide mechanisms to drill down into congested areas. Another consideration is that by showing each time step independently, the approach still requires the viewer to reconcile the structural differences – it doesn't inherently emphasize trends the way a streamgraph does. It's possible to overlay additional information (for instance, color-coding nodes by whether their values went up or down since last time, or adding a small sparkline in each node to indicate its temporal trend) to bring in some temporal summary, but fundamentally these views focus on structural snapshots. They answer questions like "What changed between these two specific steps?" or "Where did this node move over time?" more directly than the other methods. In summary, graph sequence approaches treat time as an external index to be navigated (in an animation) or juxtaposed (in a static or timeline view), rather than embedding time into the visualization encoding itself. They provide the clearest view of each individual state of the tree (crucial for tasks requiring exact topology or detailed value reading at specific times) and can illustrate evolution by explicit comparison. The trade-off is that they can demand more from the user in terms of effort to piece together the narrative of change, especially in an animation scenario, and they can suffer from scalability issues in a static view if the data is too large or the sequence too long. Effective systems often combine these with interaction (for example, allowing users to click on a node to highlight it across all time steps, or to get a textual summary of its changes) to help navigate the complexity.

Overall, the distinction between these three approaches – stable layouts, stream/flow-based, and sequence-based – lies in how they balance temporal continuity versus structural clarity. Stable tree layouts and streamgraphs both strive to make changes appear smooth and integrated, either by constraining movement or by encoding change in continuous shapes, which facilitates noticing overall evolution at the cost of some precision in reading exact structures. Sequence-based methods, conversely, ensure each state is depicted with high fidelity, enabling precise analysis of structure at specific moments, but they rely more on user cognition or explicit linking to piece together the story over time. An effective visualization of evolving hierarchies may even combine elements of these approaches, using stable animation for gradual transitions, streams for high-level trends, and small multiples or timeline views for detailed comparisons, depending on the analysis needs at hand. Each approach offers a different perspective on the data, and understanding their relative strengths and limitations is important when choosing how to visualize a given evolving tree structure. The related work in this area highlights that there is no one-size-fits-all solution – rather, a rich design space exists for capturing the dynamic nature of tree data, with each technique illuminating certain aspects of what changes, how, and when in the underlying hierarchy.

Acknowledgments

The authors wish to thank A, B, and C. This work was supported in part by a grant from XYZ (# 12345-67890).

References

[1] S. Bremm, T. von Landesberger, M. Hess, T. Schreck, P. Weil, and K. Hamacher. Interactive visual comparison of multiple trees. In *6th IEEE Conference on Visual Analytics Science and Technology, IEEE VAST 2011, Providence, RI, USA, October 23-28, 2011*, pp. 31–40. IEEE Computer Society, 2011. doi: 10.1109/VAST.2011.6102439 3

[2] M. Burch, C. Vehlow, F. Beck, S. Diehl, and D. Weiskopf. Parallel edge splatting for scalable dynamic graph visualization. *IEEE Trans. Vis. Comput. Graph.*, 17(12):2344–2353, 2011. doi: 10.1109/TVCG.2011.226 3, 4

[3] L. Byron and M. Wattenberg. Stacked graphs - geometry & aesthetics. *IEEE Trans. Vis. Comput. Graph.*, 14(6):1245–1252, 2008. doi: 10.1109/TVCG.2008.166 2

[4] S. K. Card, B. Suh, B. A. Pendleton, J. Heer, and J. W. Bodnar. Time tree: Exploring time changing hierarchies. In *2006 IEEE Symposium On Visual Analytics Science And Technology*, pp. 3–10. IEEE, 2006. 3

[5] E. Cuenca, A. Sallaberry, F. Y. Wang, and P. Poncelet. Multistream: A multiresolution streamgraph approach to explore hierarchical time series. *IEEE Trans. Vis. Comput. Graph.*, 24(12):3160–3173, 2018. doi: 10.1109/TVCG.2018.2796591 2

[6] W. Cui, S. Liu, Z. Wu, and H. Wei. How hierarchical topics evolve in large text corpora. *IEEE Trans. Vis. Comput. Graph.*, 20(12):2281–2290, 2014. doi: 10.1109/TVCG.2014.2346433 3

[7] J. A. G. Gómez, A. Buck-Coleman, C. Plaisant, and B. Shneiderman. Treeversity: Comparing tree structures by topology and node's attributes differences. In *6th IEEE Conference on Visual Analytics Science and Technology, IEEE VAST 2011, Providence, RI, USA, October 23-28, 2011*, pp. 275–276. IEEE Computer Society, 2011. doi: 10.1109/VAST.2011.6102471 3

[8] J. A. G. Gómez, M. L. Pack, C. Plaisant, and B. Shneiderman. Visualizing change over time using dynamic hierarchies: Treeversity2 and the stemview. *IEEE Trans. Vis. Comput. Graph.*, 19(12):2566–2575, 2013. doi: 10.1109/TVCG.2013.231 3

[9] M. Graham and J. B. Kennedy. Multiform views of multiple trees. In *12th International Conference on Information Visualisation, IV 2008, 8-11 July 2008, London, UK*, pp. 252–257. IEEE Computer Society, 2008. doi: 10.1109/IV.2008.21 3

[10] K. Gray, M. Li, A. R. Ahmed, and S. G. Kobourov. Visualizing evolving trees. In P. Angelini and R. von Hanxleden, eds., *Graph Drawing and Network Visualization - 30th International Symposium, GD 2022, Tokyo, Japan, September 13-16, 2022, Revised Selected Papers*, vol. 13764 of *Lecture Notes in Computer Science*, pp. 319–335. Springer, 2022. doi: 10.1007/978-3-031-22203-0 23 2

[11] M. Greilich, M. Burch, and S. Diehl. Visualizing the evolution of compound digraphs with timearctrees. *Comput. Graph. Forum*, 28(3):975–982, 2009. doi: 10.1111/J.1467-8659.2009.01451.X 2, 3

[12] S. Hahn, J. Trümper, D. Moritz, and J. Döllner. Visualization of varying hierarchies by stable layout of voronoi treemaps. In R. S. Laramee, A. Kerren, and J. Braz, eds., *Proceedings of the 5th International Conference on Information Visualization Theory and Applications, IVAPP 2014, Lisbon, Portugal, 5-8 January, 2014*, pp. 50–58. SciTePress, 2014. doi: 10.5220/0004686200500058 2

[13] S. Havre, E. G. Hetzler, P. Whitney, and L. T. Nowell. Themeriver: Visualizing thematic changes in large document collections. *IEEE Trans. Vis. Comput. Graph.*, 8(1):9–20, 2002. doi: 10.1109/2945.981848 2

[14] W. Köpp and T. Weinkauf. Temporal treemaps: Static visualization of evolving trees. *IEEE Trans. Vis. Comput. Graph.*, 25(1):534–543, 2019. doi: 10.1109/TVCG.2018.2865265 2, 3

[15] M. Sondag, B. Speckmann, and K. Verbeek. Stable treemaps via local moves. *IEEE Trans. Vis. Comput. Graph.*, 24(1):729–738, 2018. doi: 10.1109/TVCG.2017.2745140 2

[16] S. Tak and A. Cockburn. Enhanced spatial stability with hilbert and moore treemaps. *IEEE Trans. Vis. Comput. Graph.*, 19(1):141–148, 2013. doi: 10.1109/TVCG.2012.108 2

[17] Y. Tu and H. Shen. Visualizing changes of hierarchical data using treemaps. *IEEE Trans. Vis. Comput. Graph.*, 13(6):1286–1293, 2007. doi: 10.1109/TVCG.2007.70529 2

[18] E. F. Vernier, J. L. D. Comba, and A. C. Telea. A stable greedy insertion treemap algorithm for software evolution visualization. In *31st SIBGRAPI Conference on Graphics, Patterns and Images, SIBGRAPI 2018, Paraná, Brazil, October 29 - Nov. 1, 2018*, pp. 158–165. IEEE Computer Society, 2018. doi: 10.1109/SIBGRAPI.2018.00027 2

A About Appendices

Refer to **??** for instructions regarding appendices.

B Troubleshooting

B.1 ifpdf error

If you receive compilation errors along the lines of `Package ifpdf Error: Name clash, \ifpdf is already defined` then please add a new line `\let\ifpdf\relax` right after the `\documentclass[journal]{vgtc}` call. Note that your error is due to packages you use that define `\ifpdf` which is obsolete (the result is that `\ifpdf` is defined twice); these packages should be changed to use `ifpdf` package instead.

B.2 pdfendlink error

Occasionally (for some LaTeX distributions) this hyper-linked bibTeX style may lead to **compilation errors** (`pdfendlink ended up in different nesting level ...`) if a reference entry is broken across two pages (due to a bug in `hyperref`). In this case, make sure you have the latest version of the `hyperref` package (i.e. update your LaTeX installation/packages) or, alternatively, revert back to `\bibliographystyle{abbrv-doi}` (at the expense of removing hyperlinks from the bibliography) and try `\bibliographystyle{abbrv-doi-hyperref}` again after some more editing.